SyPy
Security & Privacy
Research Group

Lecture 4
Basics of ML (Part-2)

Prof. Ankit Gangwal
Assistant Professor, IIIT-H, India
email: *gangwal@iiit.ac.in*
Web: *CiaoAnkit.github.io*

Mini Quiz

Mini Quiz

not really, just a question

Basics of Machine Learning

Question

- Without using calculators, evaluate the following:

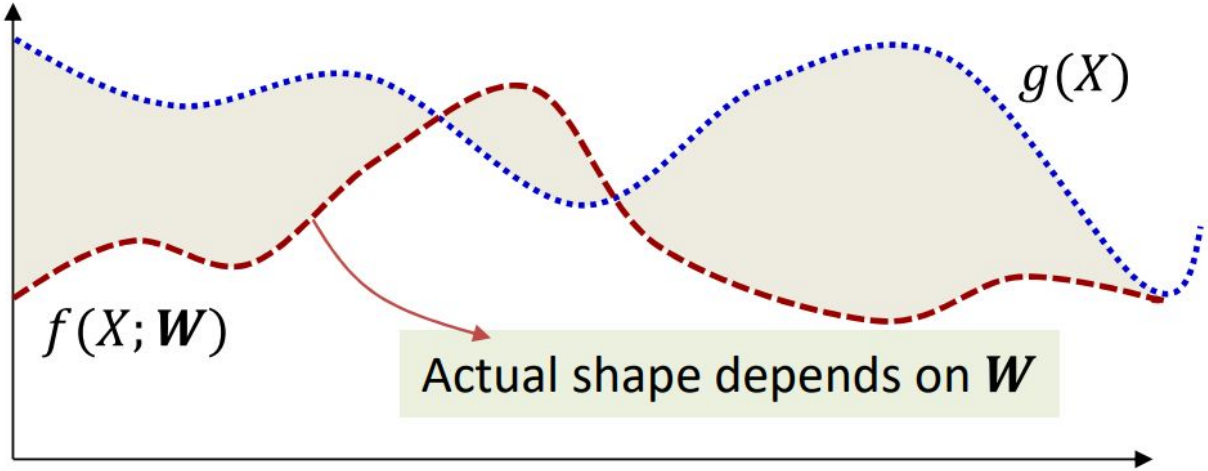
$$\frac{\pi}{4} + \cos\left(\frac{\pi}{4}\right) + \cos\left(\frac{\pi}{4} + \cos\left(\frac{\pi}{4}\right)\right) + \cos\left(\frac{\pi}{4} + \cos\left(\frac{\pi}{4}\right) + \cos\left(\frac{\pi}{4} + \cos\left(\frac{\pi}{4}\right)\right)\right)$$

Recap

Recap

Neural Networks

- Revisiting the perceptron
  - Let us attempt to construct boolean gates!
- Revisiting the network of perceptrons
  - Why the focus on “granularity”?
  - Remember this:



Recap

Entropy and Loss

SyPy Security & Privacy Research Group

- Revisiting the information ~~dump~~ theoretic concepts
  - Information Gain
  - Entropy
  - Regarding the doubt from the last class…
- What is loss?

Feedback

Feedback

SyPy
Security & Privacy
Research Group

I did not like the way the class was structured / delivered on 17th Jan. The TA Raghav seemed a bit fixated on a few details but was brushing over what the perceptron actually was, and I do not believe my doubt regarding the structure of the neural network was properly cleared by the TA during the lecture (a quick clarification which the prof turned back and gave me during the class did help significantly). Additionally, the contents seemed quite scattered. It initially made sense, but then the detour to information theory and entropy seemed completely random to the prior parts of the lecture overall. While I did understand what was being taught, the flow was not made clear at all.

If any of the answers I give do not satisfy you, please make sure to let me know… (Either right there or even after class)

As for the contents being scattered, it might seem like that but these are just basics. It will all make sense as we progress in the course

Feedback

SyPy
Security & Privacy
Research Group

Loving the course, but I do want to mention these classes happen in the evening and most of us are kinda tired, so I personally couldn't follow much in the last 20 minutes of the class, and the class time extended a bunch too. Would be great if the classes were confined to 1hr 20 minutes or some time around that (preferably lower lol)
- Candice

Noted, we'll try to pace things slowly towards the end of the lecture.

We will also try our best to end the class on time. If after today also you think the lectures are too fast-paced, do let us know

SyPy Security & Privacy Research Group

Table of Content

- Model Training
- Classification Problem Formulation
- Gradients
- Gradient Ascent/Descent
- Backpropagation
- Performance Metrics (Homework!)

- So how do we train a model?

Basics of Machine Learning

MLP/ANN training

SyPy Security & Privacy Research Group

- Training an ANN (and nearly all modern DNNs) boils down to the task of minimizing the loss/cost function associated with it
- There are many sophisticated optimization techniques that use Linear Algebra and (Matrix) Calculus to train ANNs
- DNN optimization is a field in itself, and understanding all important optimization techniques is beyond the scope of this course
- Regardless, we will look at one important optimization technique that forms the basis to many other techniques, called **Gradient Descent** or **Stochastic Gradient Descent (SGD)**
- Along with it, we will also see how SGD is practically implemented using **Backpropagation**!

Basics of Machine Learning

Classification Problem Formulation

SyPy Security & Privacy Research Group

- we have a cost function $C(X)$, where X = training data and X is $n \times d$ dimensional matrix containing n datapoints of dimension d.
- MNIST is a large image dataset of 70,000 (60k train + 10k test) handwritten numbers, where each image has 28 × 28 pixels.
- In case of MNIST, d = 28 × 28 = 784 and n = 60,000

Basics of Machine Learning

Classification Problem Formulation

SyPy Security & Privacy Research Group

- We define x^i as the ith datapoint in the dataset. $\therefore X = [x^1\ x^2\ x^3 \dots x^n]^T$

  and define $c(x^i)$ the cost per example/datapoint

  which is $c(x^i) = \sum_{k=1}^{m} (y_k^i - \hat{y}_k^i)^2$, here $\hat{y}^i$ (a vector of dimension m) is the prediction of the neural network

  of the data point x^i and y^i (also a vector of dimension m) is the actual label that of the datapoint x^i. (meaning $\hat{y}^i = f(x^i)$)

- In case of MNIST, since there are 10 digits to predict (0, 1, 2, … , 9), m = 10.

- Now, we define $C(\mathcal{X}) = \frac{1}{n}\sum_{i=1}^{n} C(x^i) =$ average cost over all training dataset $=$ Total cost of the network.

$\therefore$ $C(\mathcal{X}) =$ Total cost of the network $= \frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{m}(y_k^i - \hat{y}_k^i)^2$

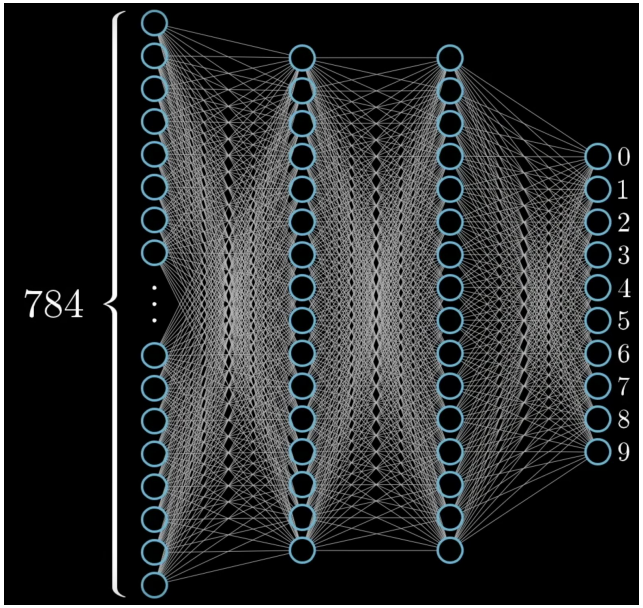
Here $\hat{y}^i = f_\theta(x^i)$, meaning f_θ is our neural network parameterized by θ that takes in input x^i gives $\hat{y}^i$ as output.

Here $\theta =$ weights and biases.

Basics of Machine Learning

Classification Problem Formulation

- In the following image
  - 784 neurons as an input layer
  - 2 hidden layers each consisting of 16 neurons
  - Output layer consisting of 10 neurons



$$\therefore \text{ Total number of weights} = \overbrace{16 \times 784}^{\text{(going from Input to 1st hidden layer)}} + \overbrace{16 \times 16}^{\text{(1st to 2nd hidden layer)}} + \overbrace{16 \times 10}^{\text{(2nd to output)}} = 12{,}960$$

$$\text{and total bias terms} = \underbrace{16 + 16}_{\text{for hidden layer neurons}} + \underbrace{10}_{\text{for output layer neurons}} = 42$$

$\therefore$ In total, there are $12,960 + 42 = 13,002$ weights and biases

θ is a collection of all these weights and biases

So we can write $\hat{y}^i$ as $\hat{y}^i = f(x^i; w_0, w_1, \ldots, w_{13,001})$

and the cost of the network as $C(X; w_0, w_1, w_2, \ldots w_{13,001})$

Basics of Machine Learning

Gradient

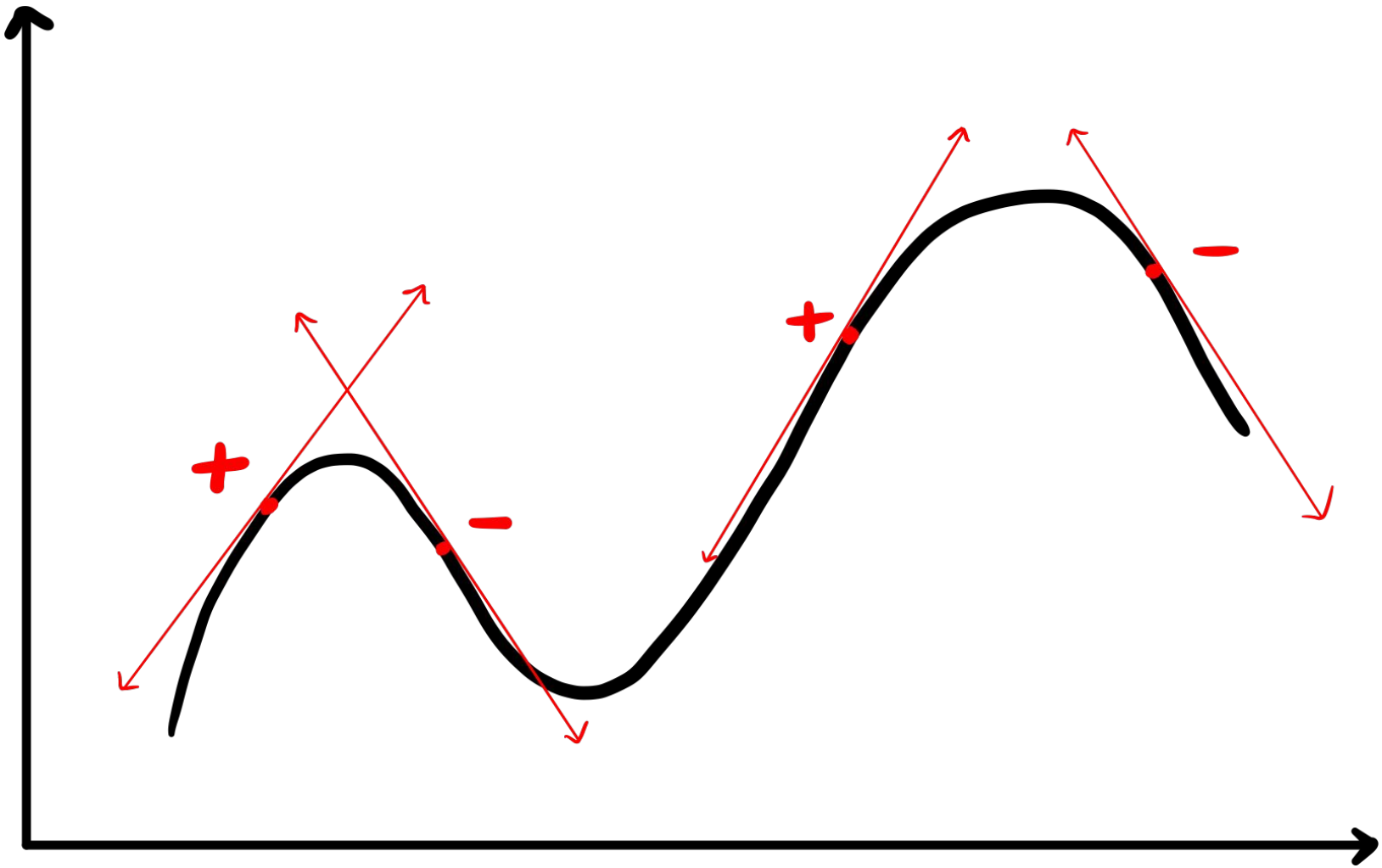
SyPy Security & Privacy Research Group

- Assume there was only one parameter instead of 13,002.
- How would you minimize the loss?
  - set derivative w.r.t. parameter = 0 and solve
  - anything else?
- Does derivative give any other information?

Basics of Machine Learning

Gradient

- Notice something? **The sign always points towards maximum!**



SyPy Security & Privacy Research Group

Gradient

- Idea: to maximize (minimize) a function, instead of taking derivative and setting it to zero, how about nudging the parameter in (opposite to) the direction of the derivative?
- This is **Gradient Ascent/Descent**!

Basics of Machine Learning

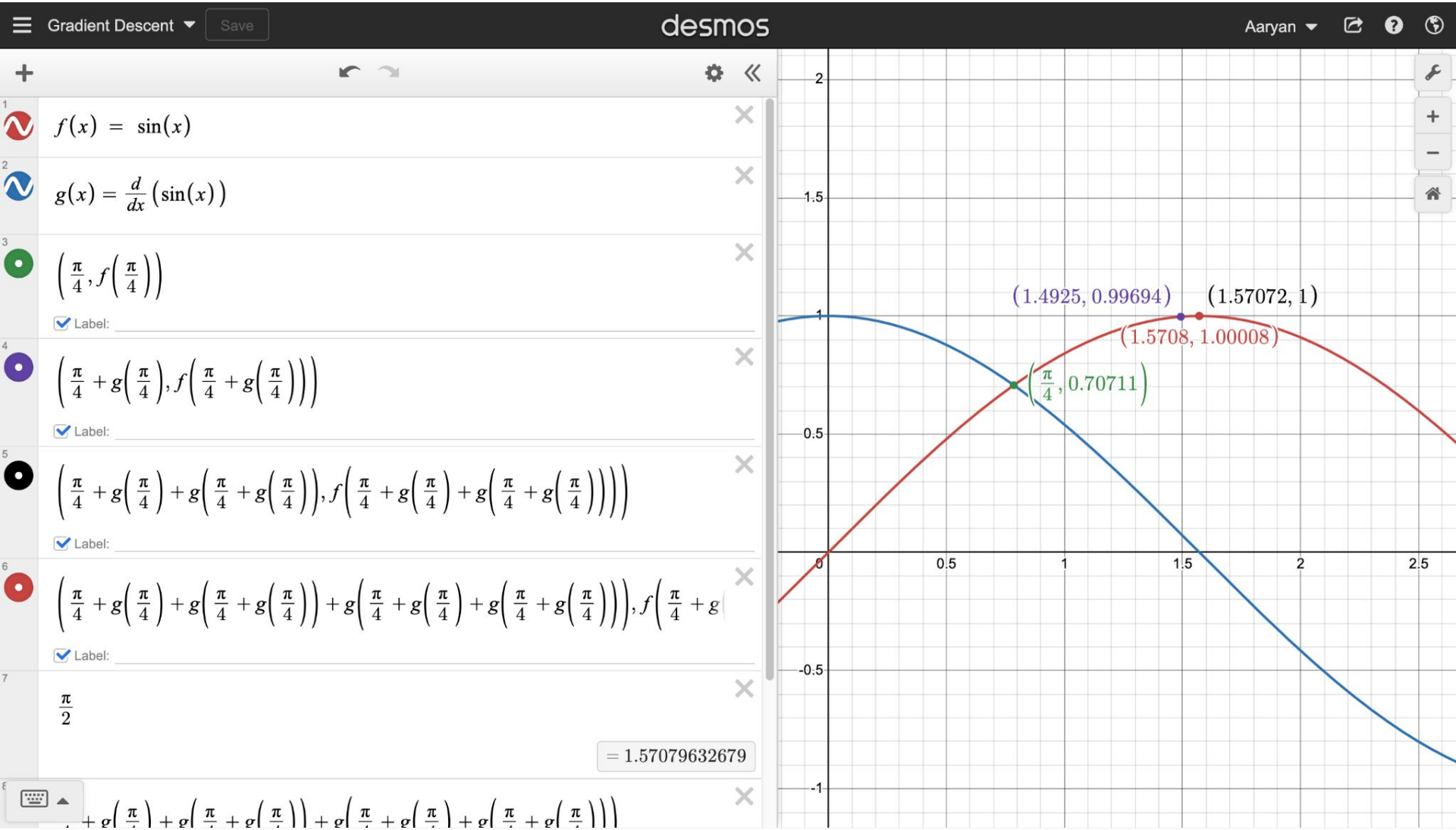
Question

- Ring a bell?

$$\frac{\pi}{4} + \cos\left(\frac{\pi}{4}\right) + \cos\left(\frac{\pi}{4} + \cos\left(\frac{\pi}{4}\right)\right) + \cos\left(\frac{\pi}{4} + \cos\left(\frac{\pi}{4}\right) + \cos\left(\frac{\pi}{4} + \cos\left(\frac{\pi}{4}\right)\right)\right)$$

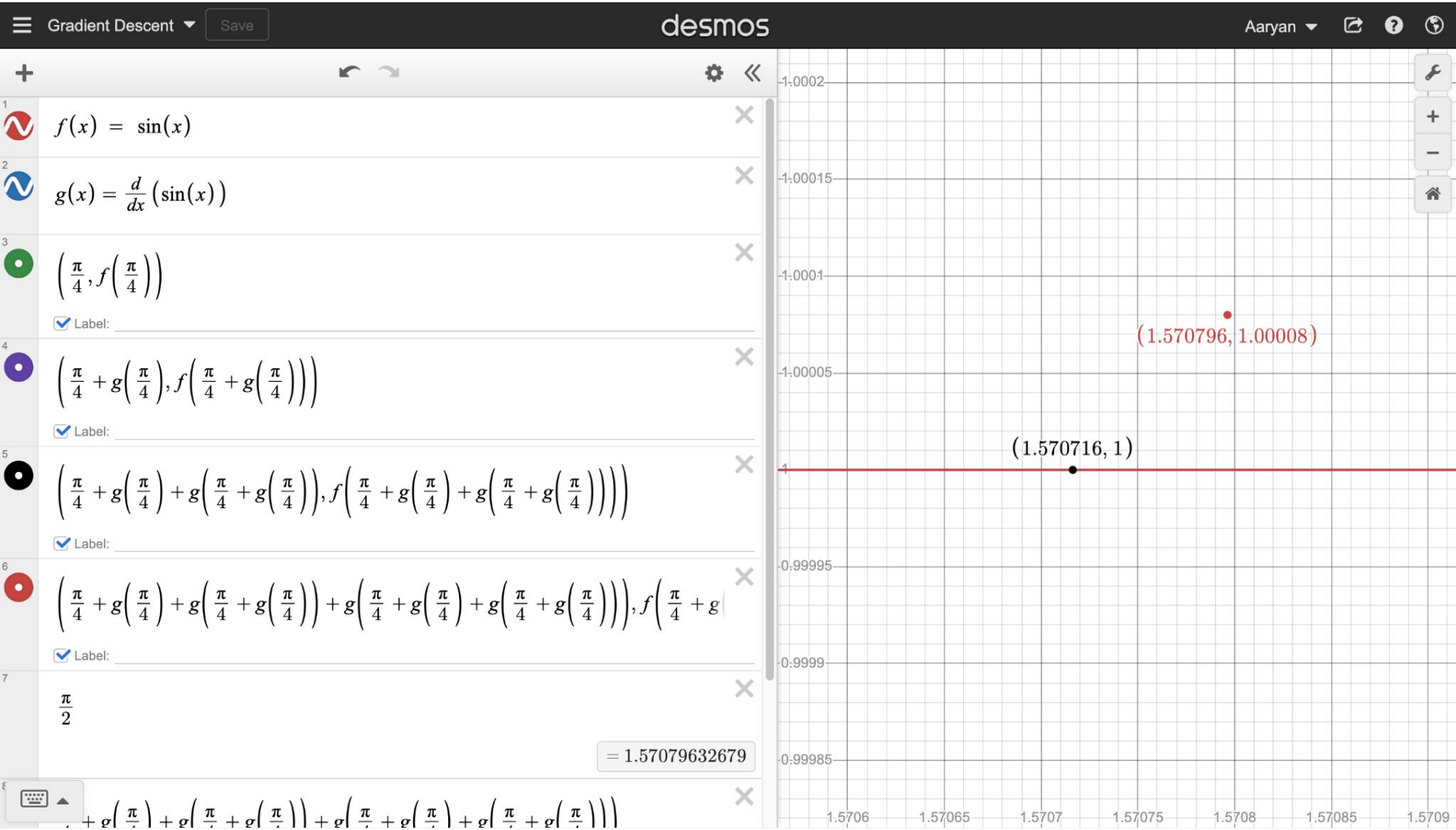
Basics of Machine Learning

Question



Basics of Machine Learning

Question



Basics of Machine Learning

Question

7

$$\frac{\pi}{2}$$

$$= 1.57079632679$$

Basics of Machine Learning

Question

7

$$\frac{\pi}{2}$$

$$= 1.57079632679$$

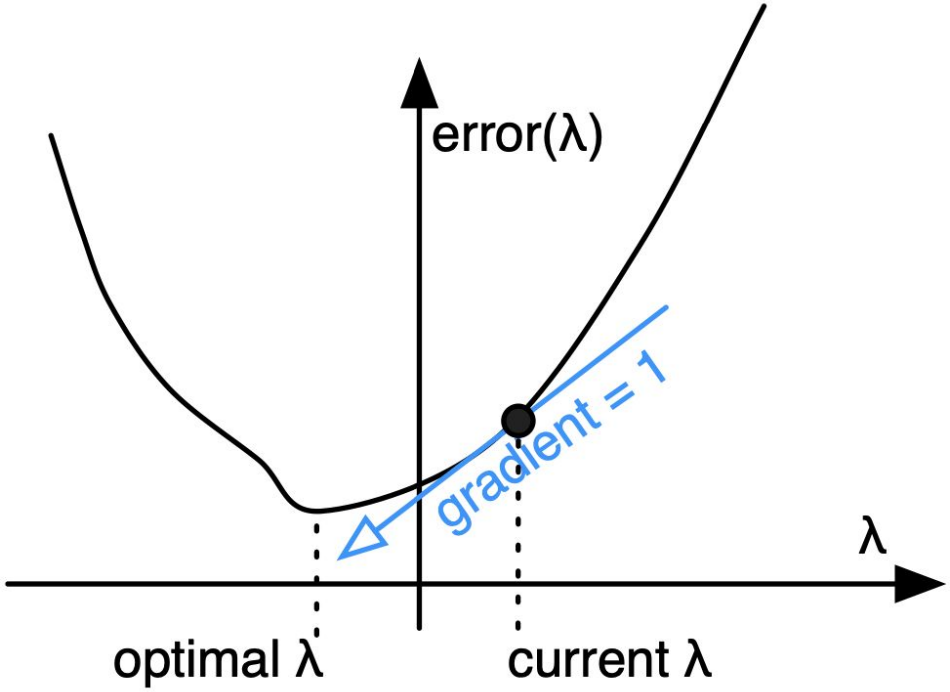
$$\frac{\pi}{4} + g\left(\frac{\pi}{4}\right) + g\left(\frac{\pi}{4} + g\left(\frac{\pi}{4}\right)\right) + g\left(\frac{\pi}{4} + g\left(\frac{\pi}{4}\right) + g\left(\frac{\pi}{4} + g\left(\frac{\pi}{4}\right)\right)\right)$$

$$= 1.57079632679$$

Link to desmos graph: https://www.desmos.com/calculator/dreexmfmqd

Basics of Machine Learning

Gradient Descent



Basics of Machine Learning

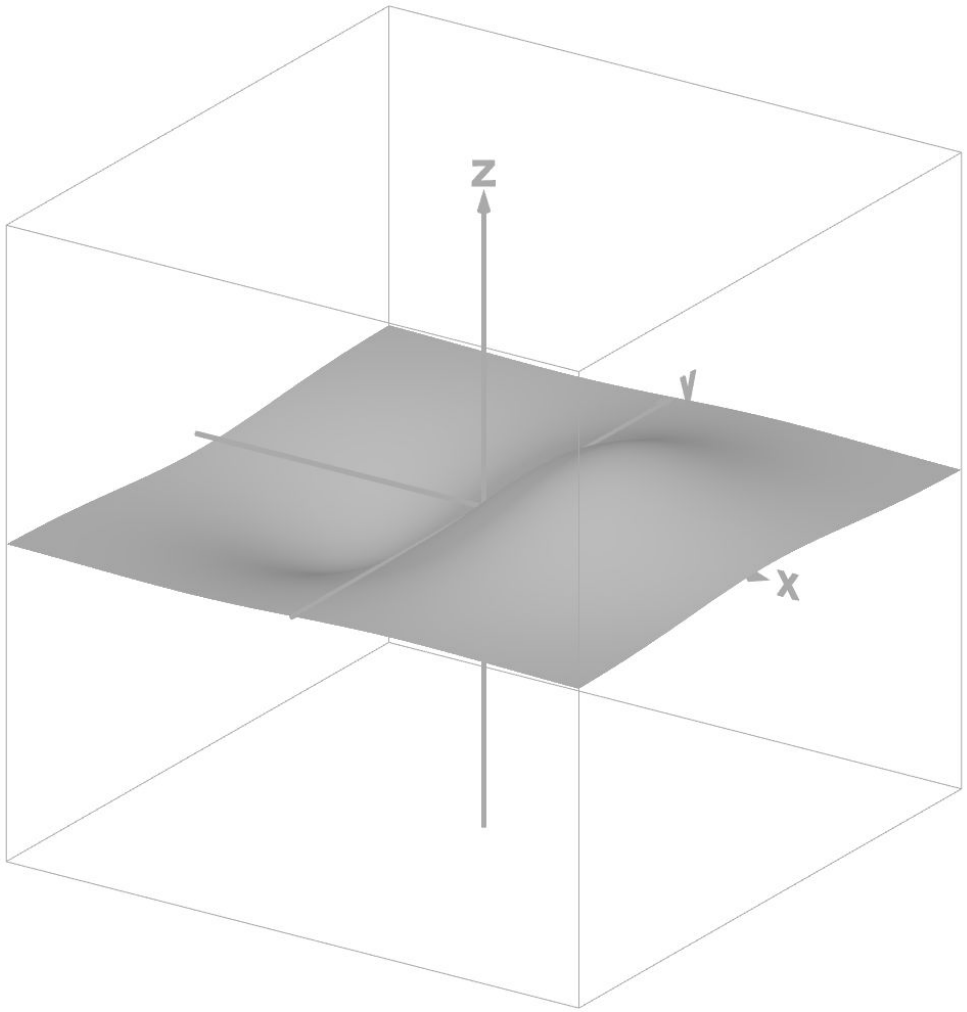
Gradient

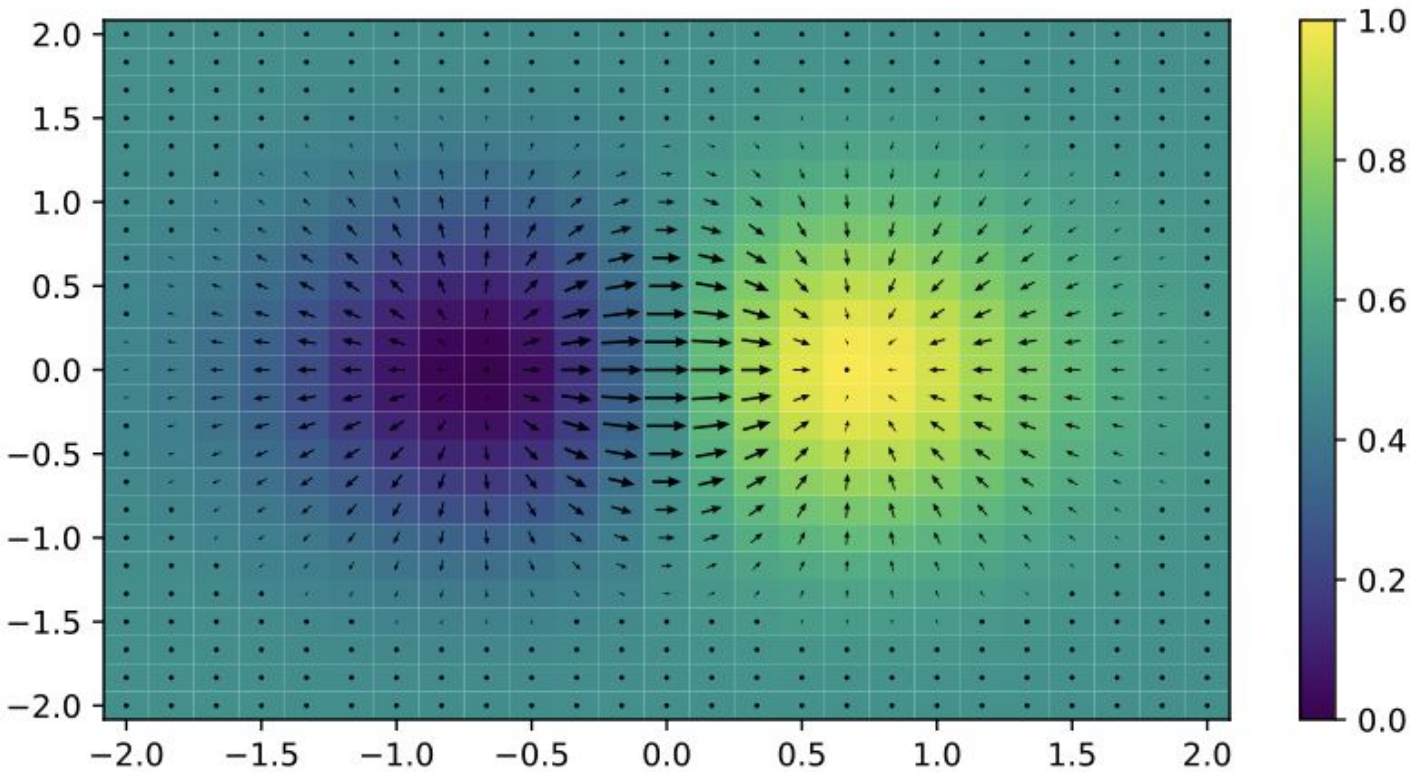
- Gradient is (loosely) a higher-order generalization of derivatives
- Similar to their 1D analogue, derivatives, that point to the direction of maximum/minimum… **gradients point towards the direction of steepest ascent!**

Gradient

A 3D example:

$$f(x, y) = xe^{-(x^2+y^2)}$$





Gradient of the 2D function $f(x, y) = xe^{-(x^2+y^2)}$ is plotted as arrows over the pseudocolor plot of the function.

SyPy Security & Privacy Research Group

From Wikipedia:

When a coordinate system is used in which the basis vectors are not functions of position, the gradient is given by the vector[a] whose components are the partial derivatives of f at p.[2] That is, for $f\colon \mathbb{R}^n \to \mathbb{R}$, its gradient $\nabla f\colon \mathbb{R}^n \to \mathbb{R}^n$ is defined at the point $p = (x_1, \ldots, x_n)$ in *n*-dimensional space as the vector[b]

$$\nabla f(p) = \begin{bmatrix} \dfrac{\partial f}{\partial x_1}(p) \\ \vdots \\ \dfrac{\partial f}{\partial x_n}(p) \end{bmatrix}.$$

Basics of Machine Learning

[Optional] How gradients gives the direction of steepest ascent

1. Why the gradient is the direction of steepest ascent
2. My notes based on the video

Gradient Descent Equation

$$\theta_{n+1} = \theta_n - \eta \nabla_\theta C(\mathcal{X}; \theta_n)$$

η is the learning rate, which decides how 'big' of a step you want to take

- If you set a big learning rate, your algorithm may not converge to the optimal solution
- If you set it too small, convergence may take forever
- Part of hyperparameter of the model -> setting an appropriate value will come with experience

SyPy Security & Privacy Research Group

Backpropagation

- we want $-\nabla_w C(X; w_0, w_1, \ldots, w_{13,001})$ so that we can (iteratively) update the weights and biases and decrease the cost of the network.

- **Backpropagation** is the algorithm for computing this crazy complicated gradient efficiently.

Basics of Machine Learning

Backpropagation

- We're going to look at the way to backpropagate using **computational graphs**
- I will be broadly following this blog (Christopher Olah's blog on backpropagation, it is an amazing resource if you're interested!)
- Most of the diagrams you will see in the upcoming section have been taken from there, credits to the blog for that

Computational Graphs

Q. What is a computational graph?

Ans. Just a nice way to visualise mathematical expressions

Q. What makes it "nice"?

Ans. It has some properties we can use to simplify visualising the flow of operations and stuff like derivatives

Example: $e = (a + b) * (b + 1)$

Decomposing this into individual additions and multiplications (and introducing intermediary variables as required), we get the following

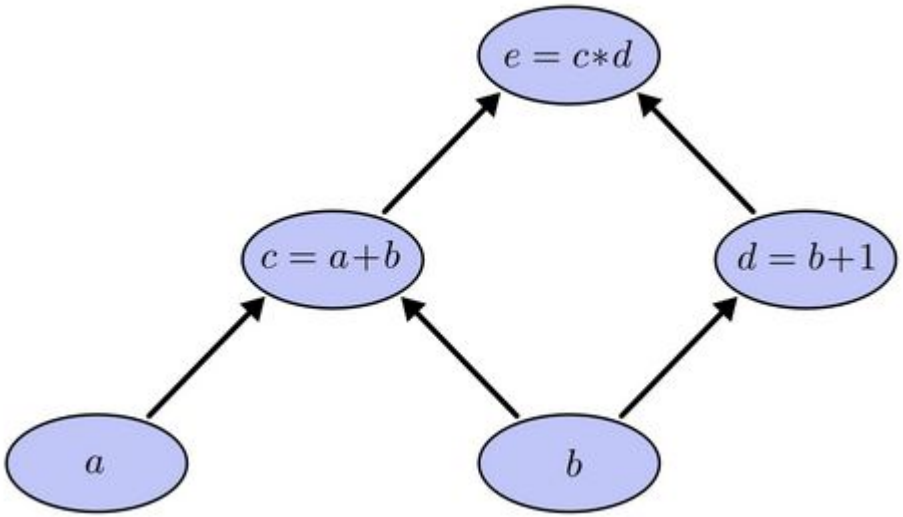
$$c = a + b$$

$$d = b + 1$$

$$e = c * d$$

Computational Graphs

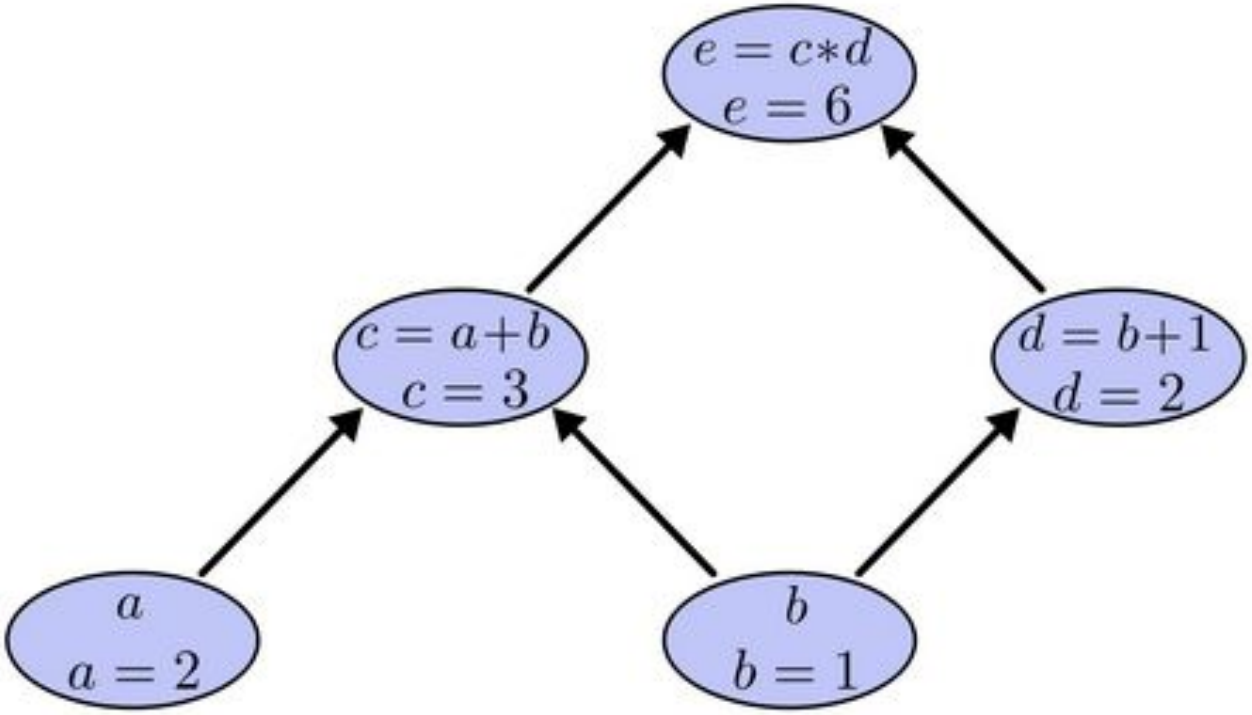
- To make a computational graph, we just need to map this sequence of operations onto a graph
- We start from the original expression and keep breaking it down into smaller, simpler parts
- Thus for this set of operations we get the following graph:



Basics of Machine Learning

Computational Graphs

- Let's try to assign values and see how the operations work out
- Set a = 2, b = 1



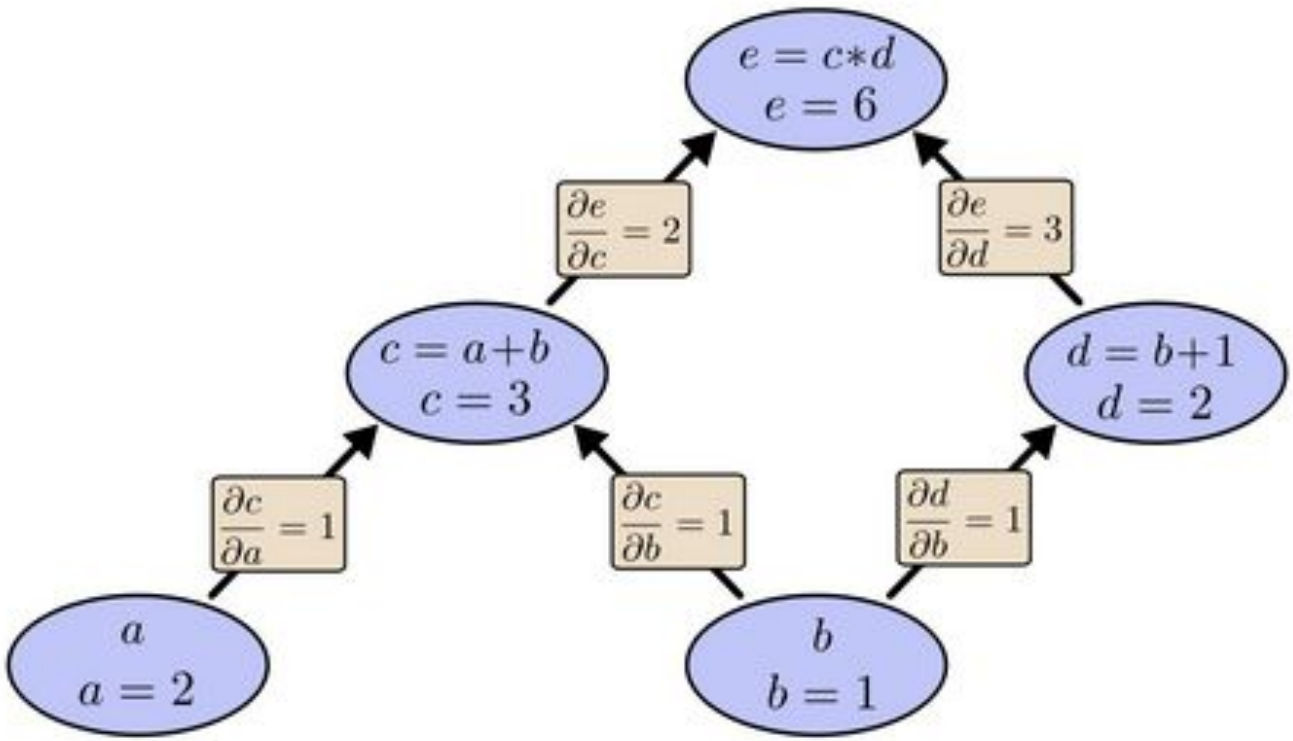
SyPy Security & Privacy Research Group

Computational Graphs

- Now how does this help us with backpropagation and derivatives?
- To see how we can differentiate using this, we need to look at the *partial derivatives* along the edges of the graph
- QUICK RECALL: Partial derivative of p w.r.t. q is where you consider the effect of only q on p, and consider the other parameters as constants
- In the example that we considered, how would we write the (partial) derivatives on the edges?

Basics of Machine Learning

Computational Graphs



Computational Graphs

- Let's try to analyse the effect between nodes that aren't directly connected to each other
- Consider how 'e' is affected by 'b'
- What we get by "summing over the paths" is this:

Computational Graphs

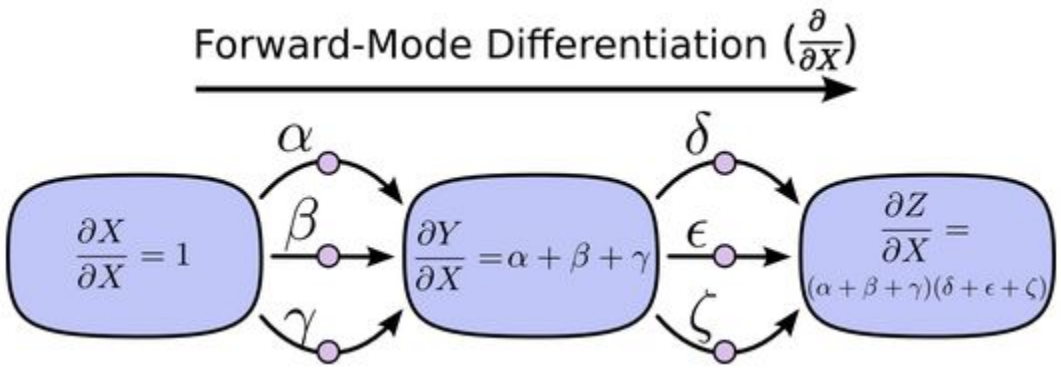
- Let's try to analyse the effect between nodes that aren't directly connected to each other
- Consider how 'e' is affected by 'b'
- What we get by "summing over the paths" is this:

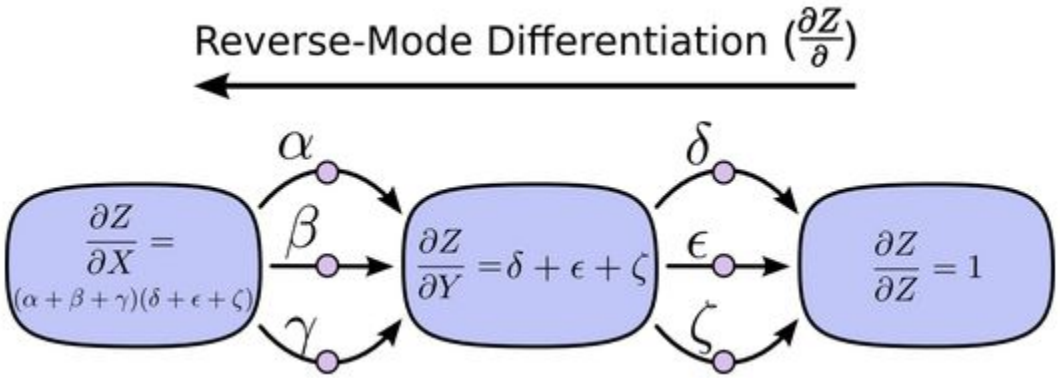
$$\frac{\partial e}{\partial b} = 1 * 2 + 1 * 3$$

- But what if there are a lot of different paths? With slightly more complex graphs you can get a much larger number of paths
- Here comes the approach of 'factoring' paths
- Two ways of summing by factoring: forward-mode differentiation and reverse-mode differentiation

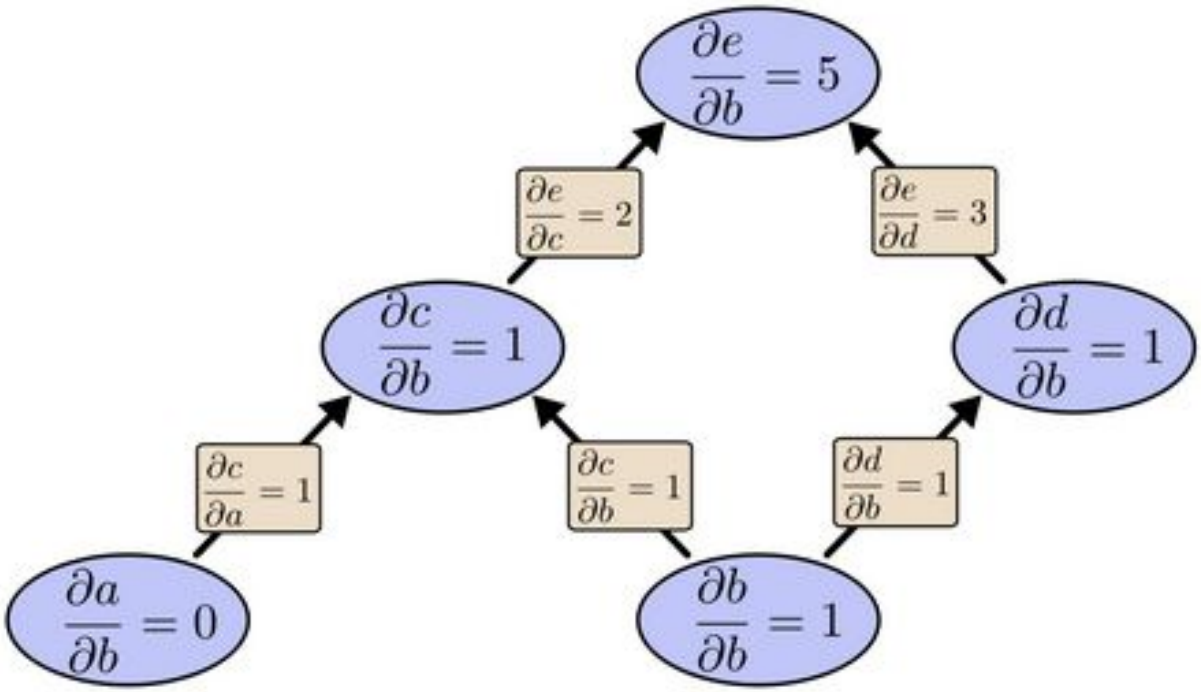
Basics of Machine Learning

Computational Graphs

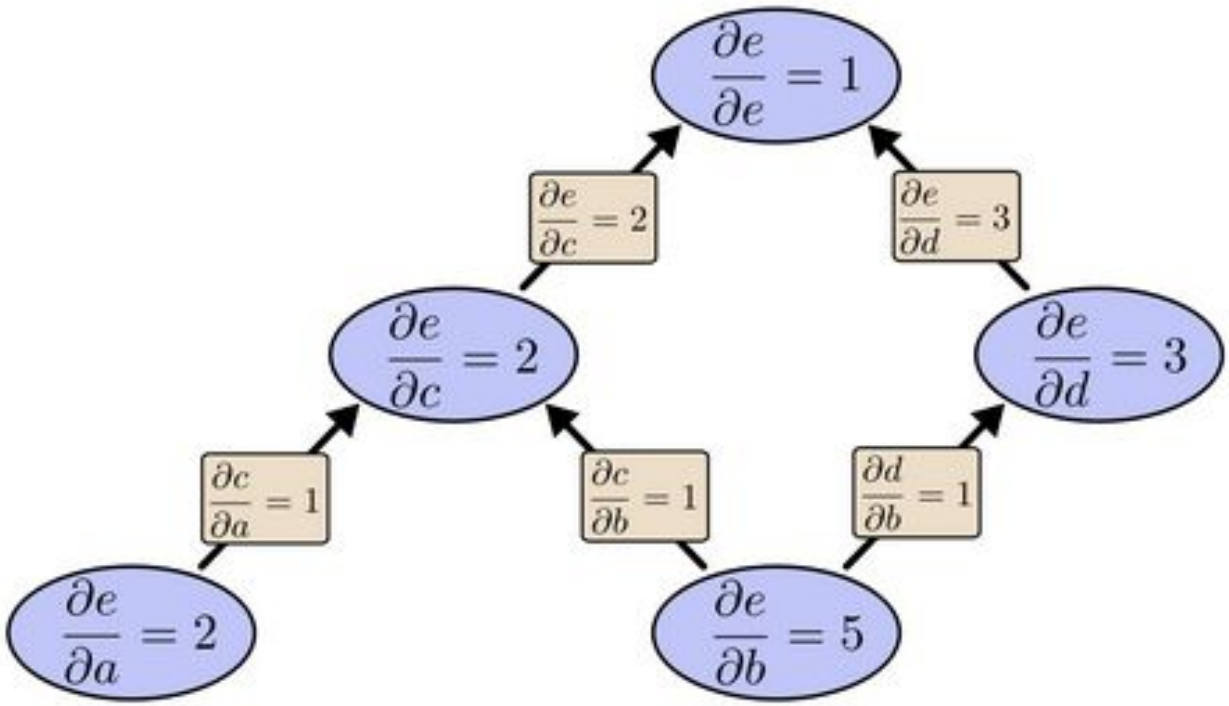




Forward-mode, from *b* up



Reverse-mode, from *e* down

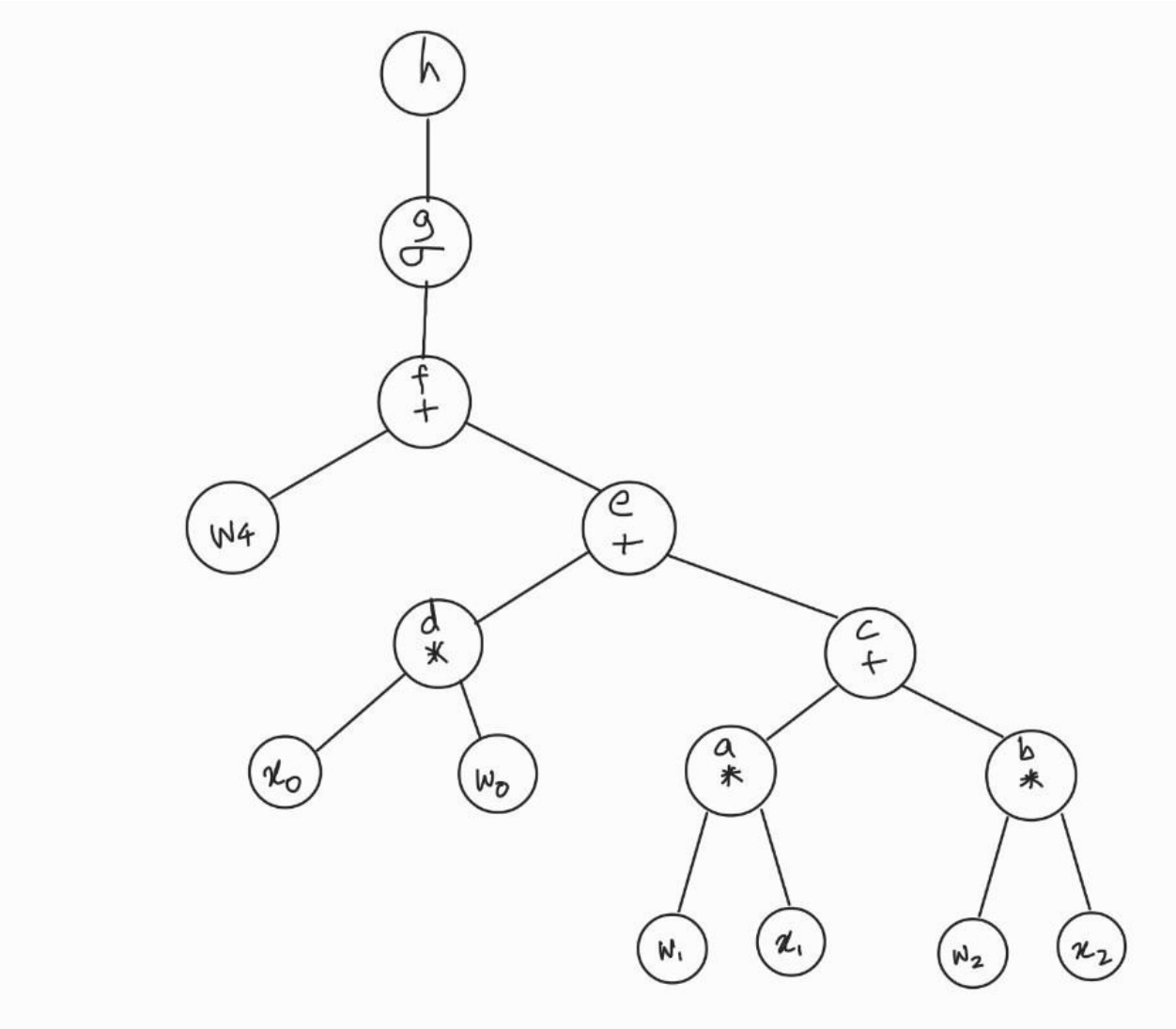


Before moving forward, let's work out another example which is a little more complex:

$$h = \sigma(w_0 x_0 + w_1 x_1 + w_2 x_2 + w_3)$$

Basics of Machine Learning

Computational Graphs



Fun exercise:

Make the graph for $(p + qr - w^2)(t\%p)$

Try to think about how you would calculate the derivative in this case! (Is it even possible?)

(Assume '%' denotes the modulo operator)

Nice detour. How does this fit in with backpropagation?

Good News!

Reverse-Mode Differentiation

is

Backpropagation!

Rejoice!

Stochastic Gradient Descent

- Can you see that you have to individually calculate the “nudge” required along the gradient for each training example?
- Averaging over all these “nudges” gives us the final gradient descent step
- We use this gradient descent step in our gradient descent algorithm
- In practice, it takes computers an extremely long time to add up the influence of every single training example, every single step
- Therefore, we will use an approximation called **Stochastic Gradient Descent (SGD)** which would speed up the process of our gradient descent algorithm

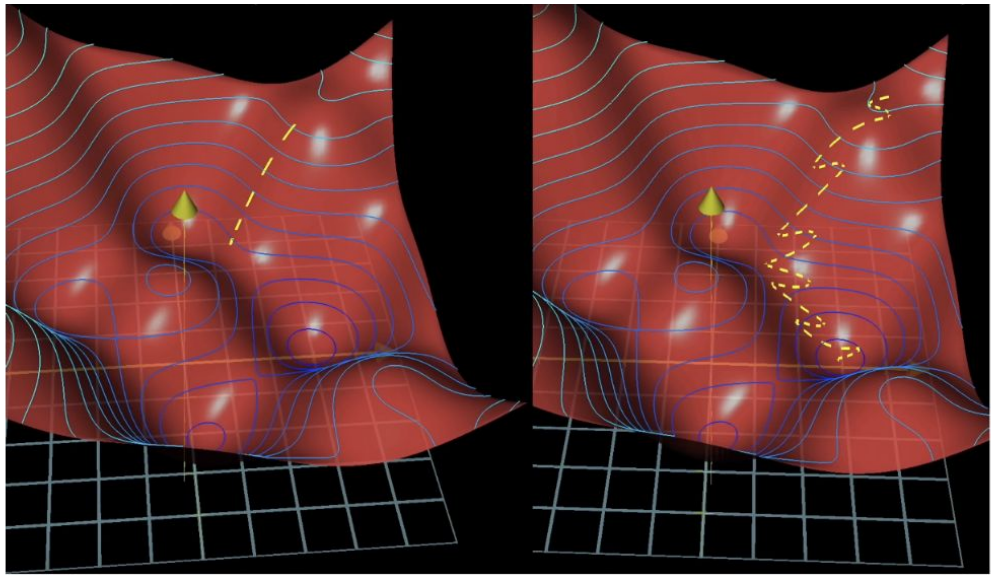
Basics of Machine Learning

Stochastic Gradient Descent

SyPy Security & Privacy Research Group

- In SGD, we randomly shuffle our training data and divide it into batches, then we backpropagate taking one batch at a time instead of the whole dataset to get our gradient descent step
- This would not be the “true” gradient descent step i.e. not the most efficient step downhill
- But each batch does give a pretty good approximation
- More importantly, it gives us a significant computational speed-up

Stochastic Gradient Descent



If gradient descent is like a wise, steady person making their way down the slope, then SGD is like a drunken man staggering wildly down it. The steps individually might not be perfectly optimal, but they are taken quickly and can help you descend much faster in practice

SyPy Security & Privacy Research Group

After you (really) understand backprop…

When I first understood what backpropagation was, my reaction was: "Oh, that's just the chain rule! How did it take us so long to figure out?" I'm not the only one who's had that reaction. It's true that if you ask "is there a smart way to calculate derivatives in feedforward neural networks?" the answer isn't that difficult.

But I think it was much more difficult than it might seem. You see, at the time backpropagation was invented, people weren't very focused on the feedforward neural networks that we study. It also wasn't obvious that derivatives were the right way to train them. Those are only obvious once you realize you can quickly calculate derivatives. There was a circular dependency.

After you (really) understand backprop…

Worse, it would be very easy to write off any piece of the circular dependency as impossible on casual thought. Training neural networks with derivatives? Surely you'd just get stuck in local minima. And obviously it would be expensive to compute all those derivatives. It's only because we know this approach works that we don't immediately start listing reasons it's likely not to.

SyPy Security & Privacy Research Group

After you (really) understand backprop…

That's the benefit of hindsight. Once you've framed the question, the hardest work is already done.

Wisdom

SyPy Security & Privacy Research Group

After you (really) understand backprop…

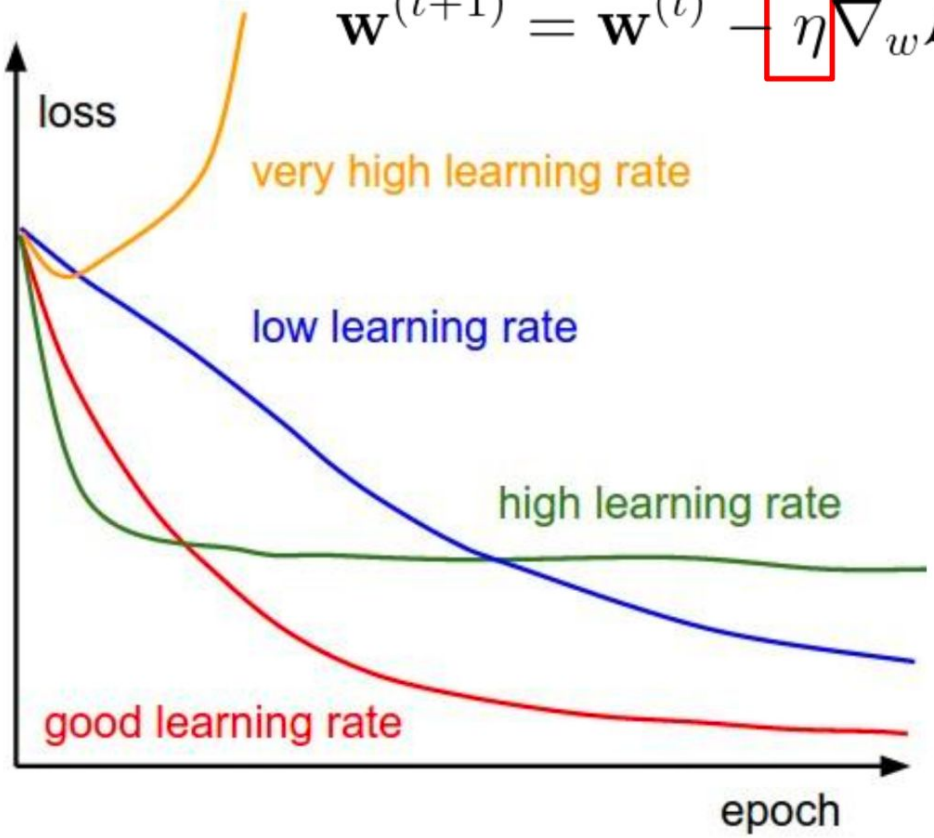
That's the benefit of hindsight. Once you've framed the question, the hardest work is already done.

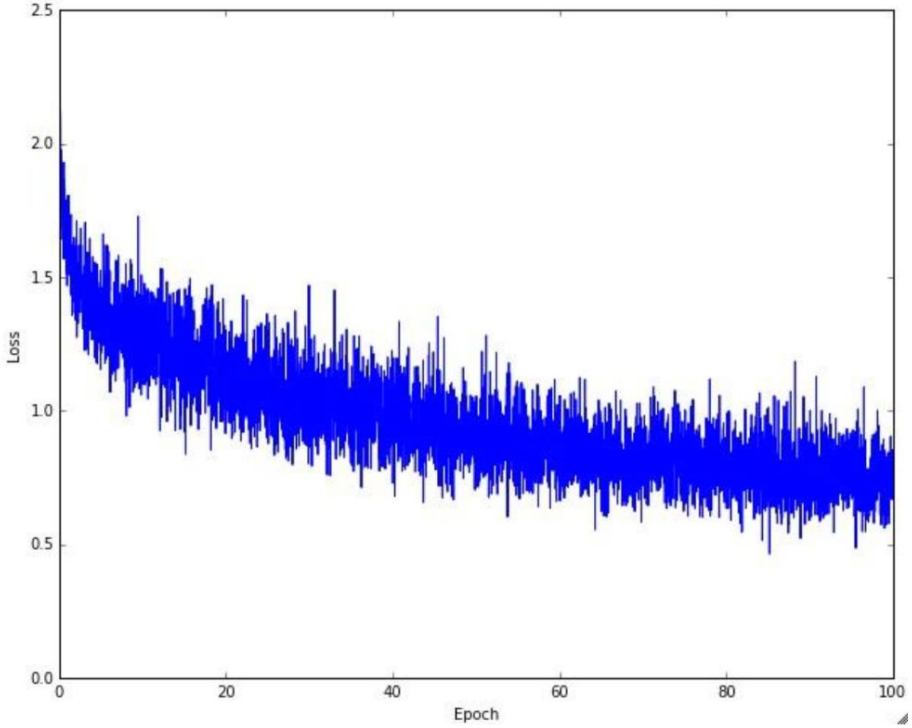
For research in general, the statement applies, and can never be overstated

Basics of Machine Learning

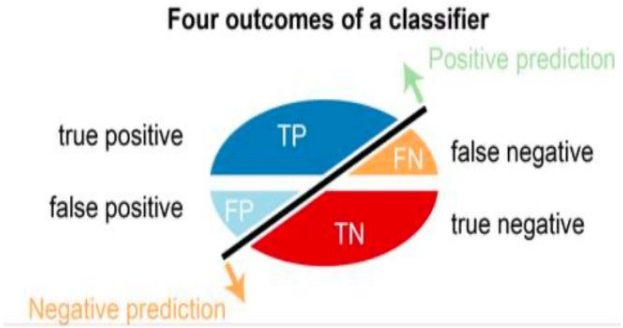
Learning rate revisited

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \boxed{\eta} \nabla_w \mathcal{L}(\mathbf{w})$$

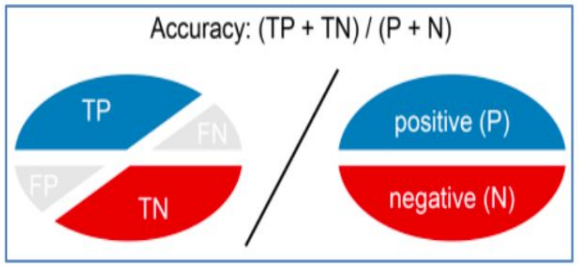




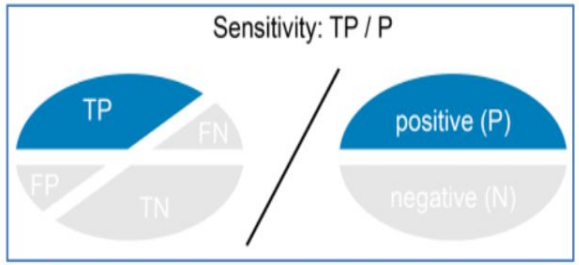
Overview of binary classification measures



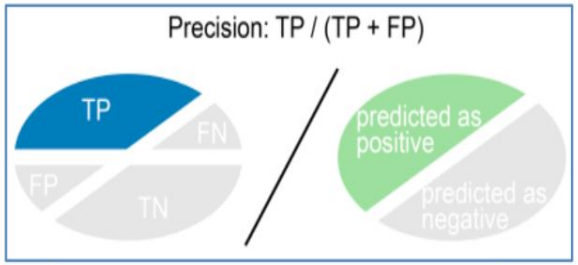
n=165	Predicted: NO	Predicted: YES	
Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
	55	110	



% of correct predictions



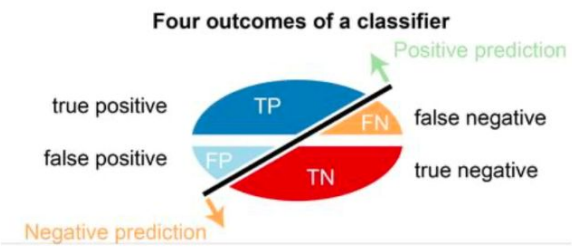
% of + class correctly predicted
[aka Recall / TPR]

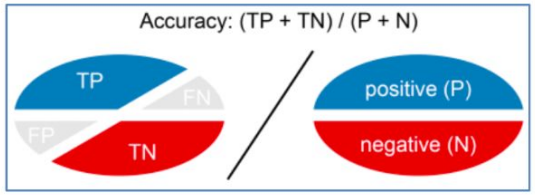


correct prediction of + class
[aka Precision]

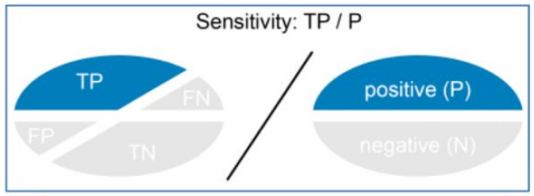
Accuracy v/s Precision v/s Recall

- Monitor **Precision** if a false positive carries higher cost.
- Monitor **Recall** if a false negative carries higher cost.

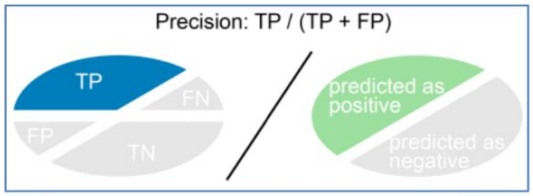




% of correct predictions



% of + class correctly predicted
[aka Recall / TPR]



correct prediction of + class
[aka Precision]

SyPy Security & Privacy Research Group

F1-Score

- What to do when one classifier has better Precision but worse Recall, while other classifier behaves exactly opposite?
  - F-measure (Information Retrieval)
    - $F_1 = \dfrac{2}{\frac{1}{Recall} + \frac{1}{Precision}}$

F1 measure punishes extreme values more !
Definition of Recall and Precision have same numerator, different denominators. A sensible way to combine them is harmonic mean.

Outro

We've covered a lot!

Take a moment to think and if you have any questions, ask now!

Basics of Machine Learning

Resources

SyPy Security & Privacy Research Group

- Basic ML tutorial using PyTorch in Jupyter Notebook, on Colab
  - Regression: https://colab.research.google.com/github/mrdbourke/pytorch-deep-learning/blob/main/01_pytorch_workflow.ipynb#scrollTo=o6rf3hTWec0_
  - Classification: https://colab.research.google.com/github/mrdbourke/pytorch-deep-learning/blob/main/02_pytorch_classification.ipynb
- Treat them as ungraded assignments
- Getting hands dirty with code is the best way to reinforce learned concepts

Basics of Machine Learning

Resources

- 3Blue1Brown Neural Network Playlist: https://youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&si=i2YikXZLc81Qa4wl
- Christopher Olah's Blog on Backpropagation: https://colah.github.io/posts/2015-08-Backprop/
- Stanford CS231n Lecture Notes on Backpropagation: https://cs231n.github.io/optimization-2/
- Andrej Karpathy's (wonderful) hands-on micrograd (Backpropagation) implementation: https://youtu.be/VMj-3S1tku0?si=rfFV8erBRkUWz6VP
- MIT IAP - Matrix Calculus for Machine Learning and Beyond: https://youtube.com/playlist?list=PLUl4u3cNGP62EaLLH92E_VCN4izBKK6OE&si=8OlrhlSo0YCf3Oyd

Feedback form

- Drop in any feedback here: https://forms.gle/smP6cs9aaDdJQeNv7